



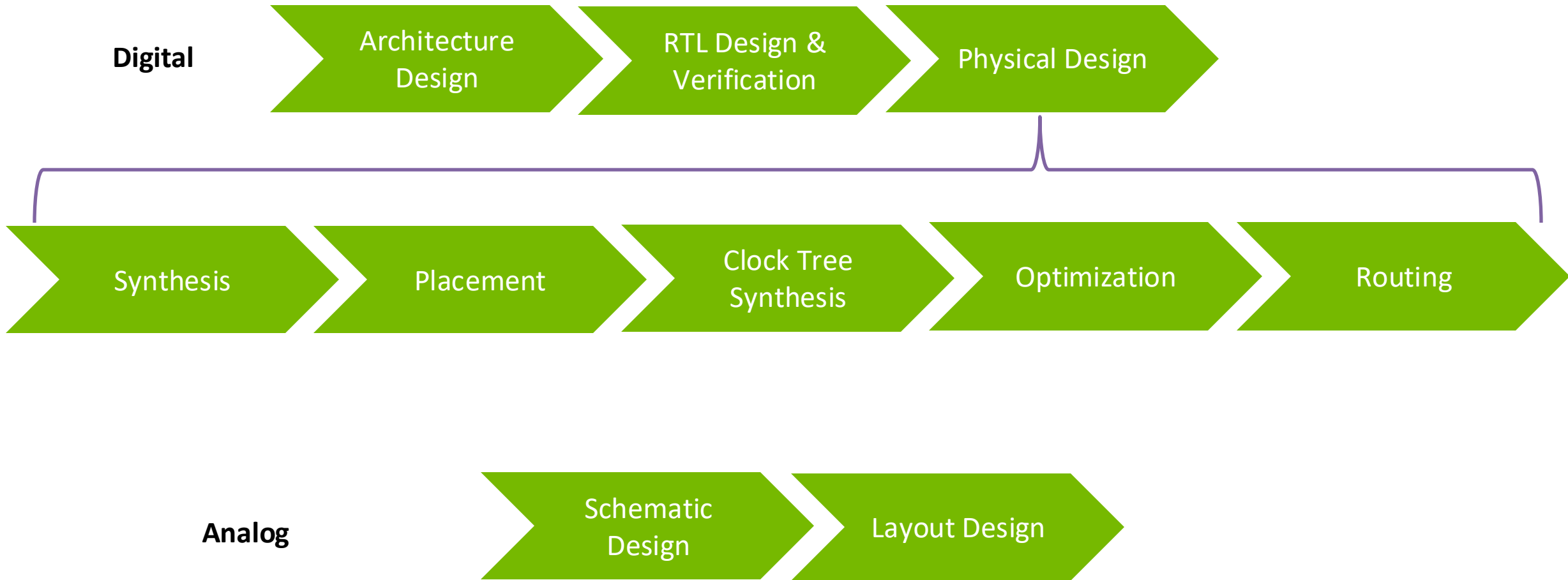
AI for Chip Design

Haoxing (Mark) Ren, Director of Design Automation Research

Research

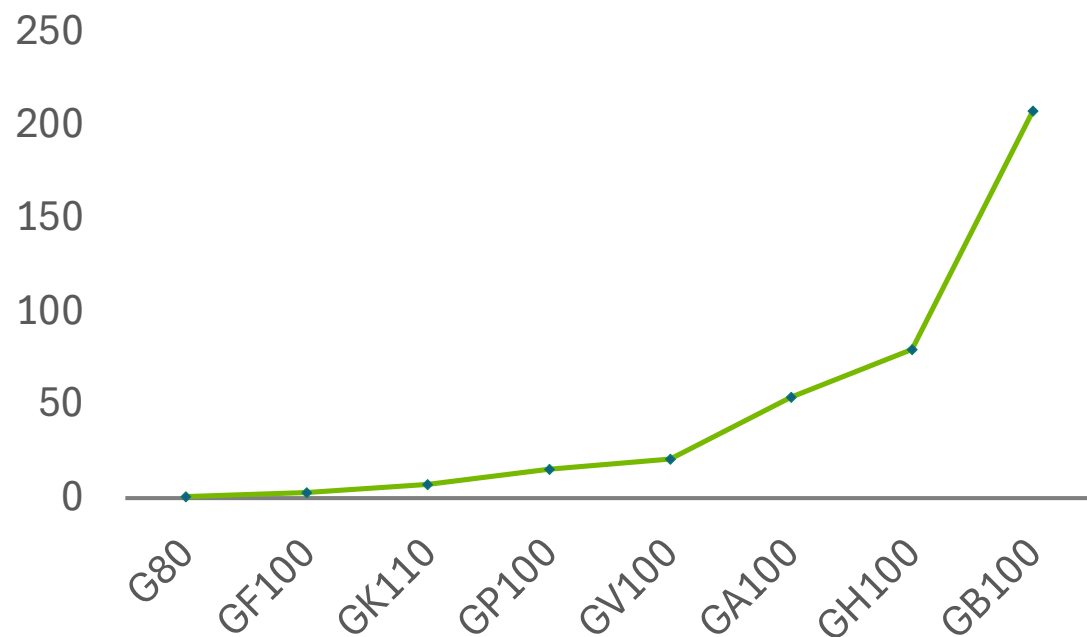
[Research Labs ▾](#)[Publications](#)[AI Playground ▾](#)[Research Areas ▾](#)[Careers ▾](#)[Licensing](#)[3D Deep Learning](#)[AI-Mediated Reality and
Interaction](#)[Applied Research](#)[Fundamental Generative AI
Research](#)[Generalist Embodied
Agents](#)[High-Fidelity Physics](#)[Autonomous Vehicles](#)[Applied Research in
Autonomous Vehicles](#)[Conversational AI](#)[Learning and Perception](#)[Perception, Action and
Reasoning](#)[Real-Time Graphics](#)[Deep Imagination](#)[Dynamic Vision and
Learning](#)[Electronic Design
Automation](#)[Robotics](#)[Taiwan Research Lab](#)

Chip design in a nutshell



Rising chip design complexity and schedule challenge

GPU TRANSISTOR COUNTS
(BILLIONS)



*It took several thousand engineers **several years** to create, at an approximate development cost of \$3 billion.*

Jensen Huang, 2017

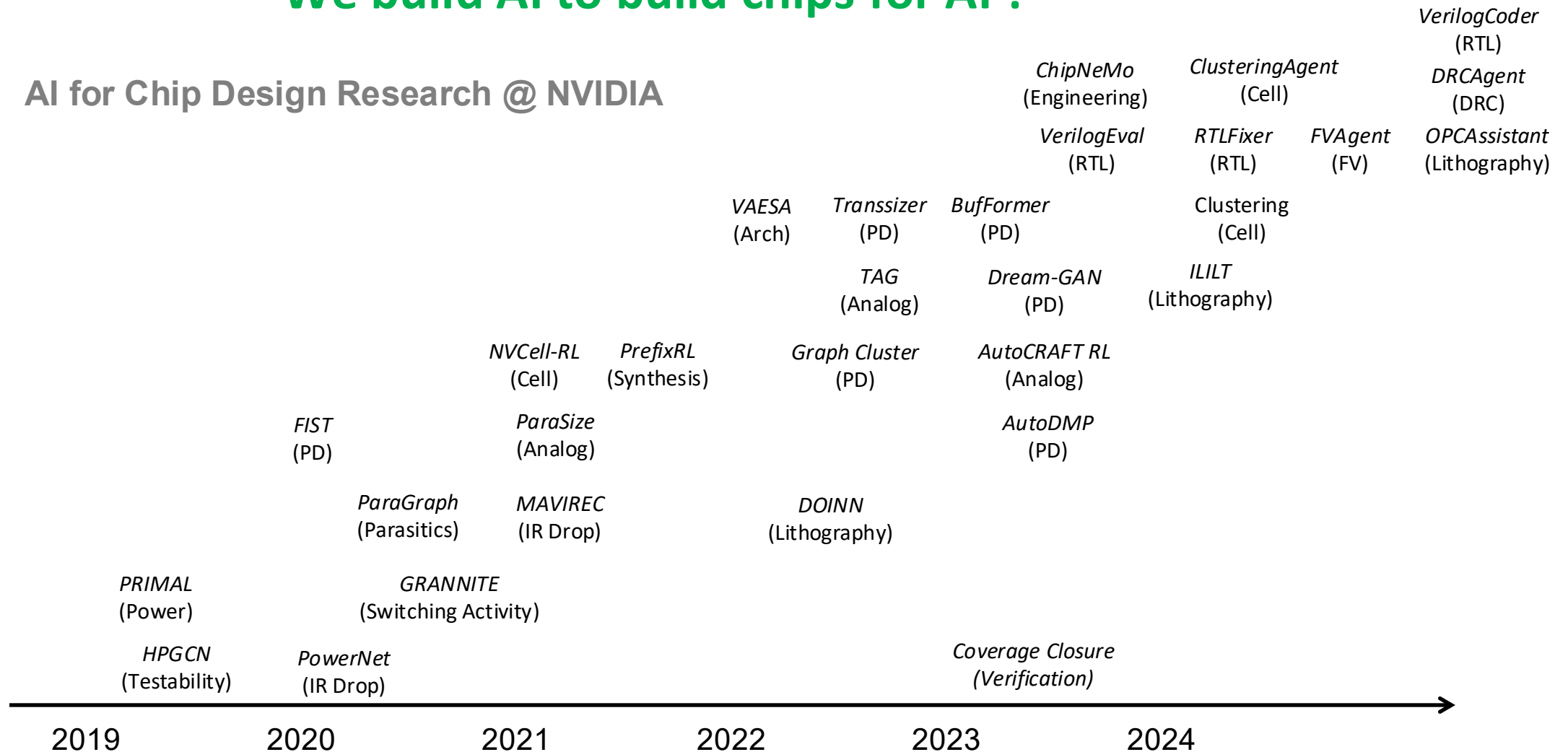
*Our company has a **one-year rhythm**. Our basic philosophy is very simple: build the entire data center scale, disaggregate and sell to you parts on a one-year rhythm, and push everything to **technology limits**,” Huang explained.*

Jensen Huang, 2024

AI improves quality and productivity of chip design

We build AI to build chips for AI !

AI for Chip Design Research @ NVIDIA



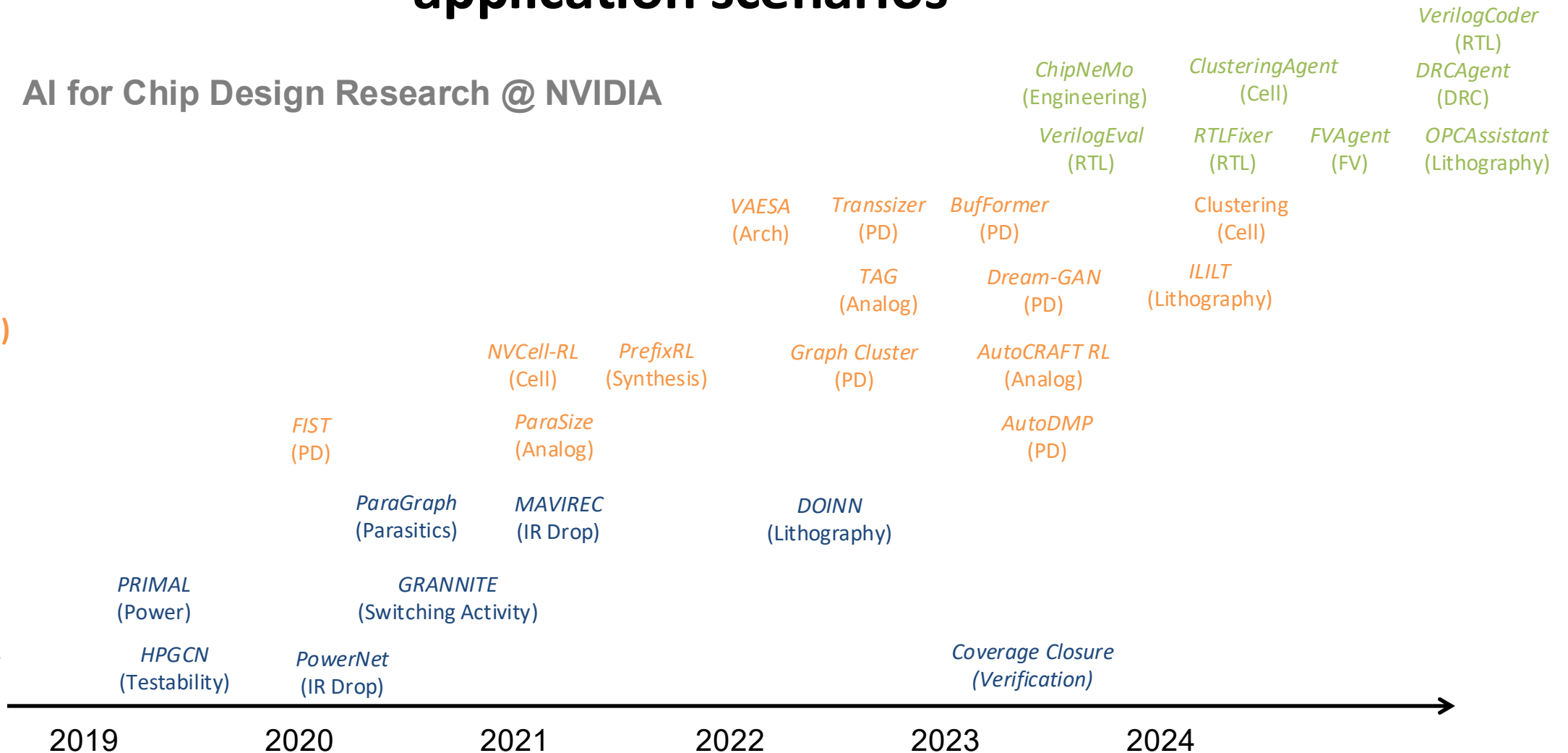
Analysis, Optimization, and Assistance are the main application scenarios

AI for Chip Design Research @ NVIDIA

**Assistance
(LLM/Agent)**
Know-how
Coding
Task automation

**Optimization
(BO/RL/GenAI)**
Faster
More scalable
Better results

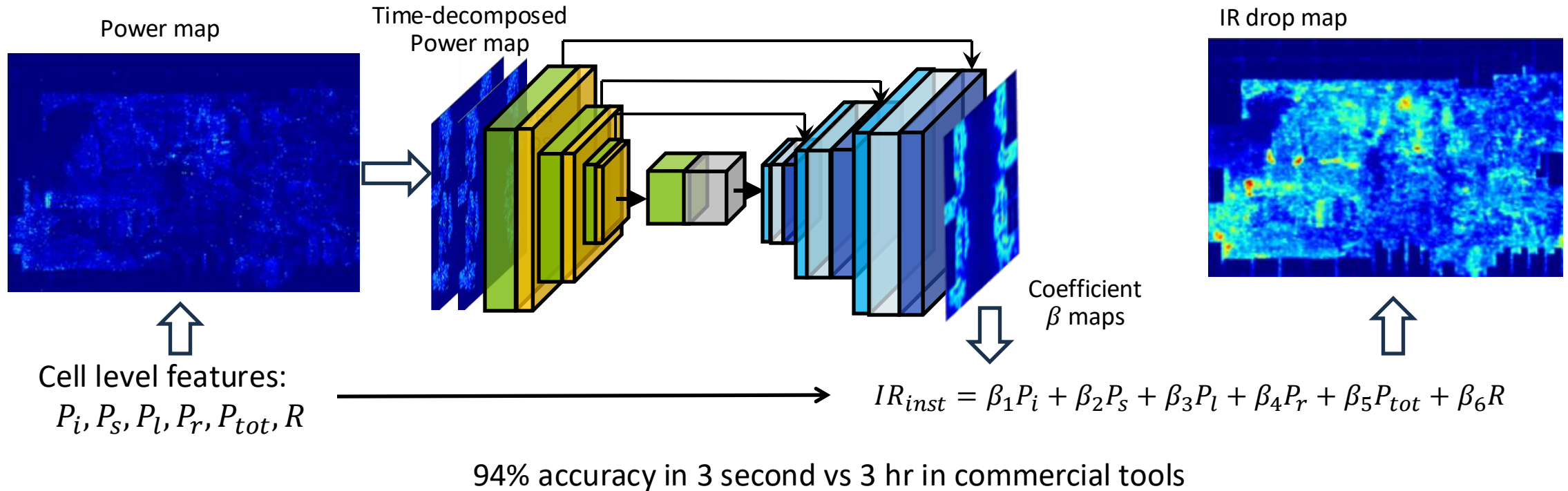
**Analysis
(CNN/GNN)**
Faster
Cross-Stage
Detect Anomaly



AI performs quick design analysis

IR drop estimation is important for physical design, but it takes hours

Use AI to predict IR drop from cell level features



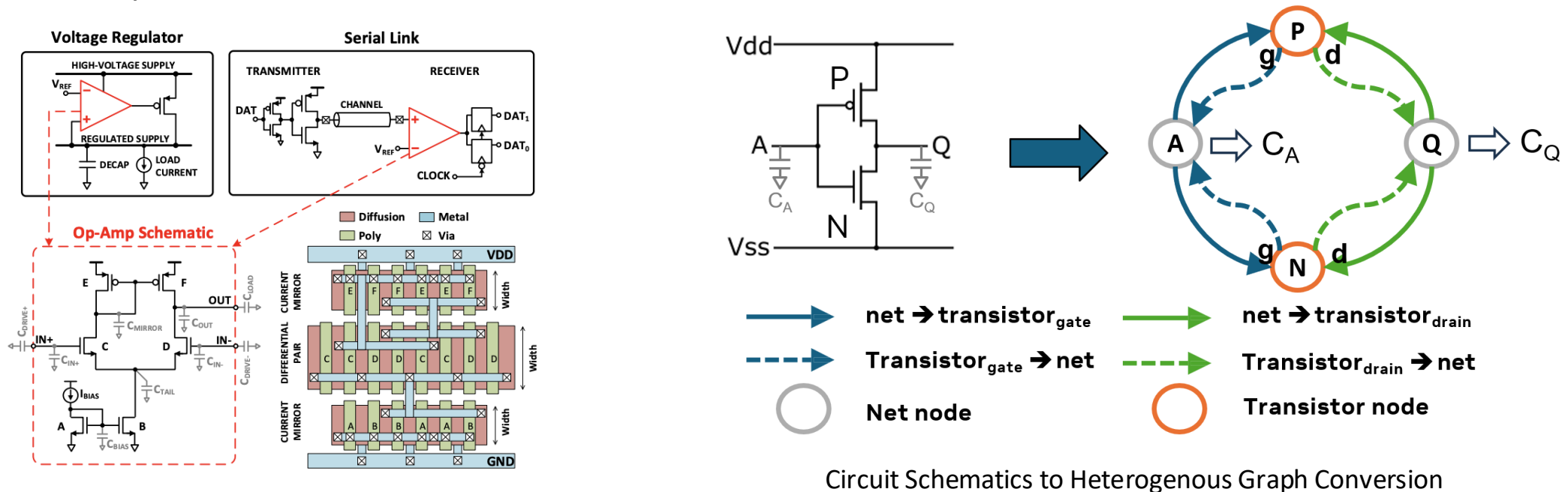
AI predicts cross-stage design metrics

Impact of layout parasitics on schematic design

Use AI to predict layout parasitics from schematic given a large dataset

Convert schematic to graph and learn with **Graph Neural Networks**

Predicted parasitics reduced simulation error to <10%



AI optimizes placement tools parameters

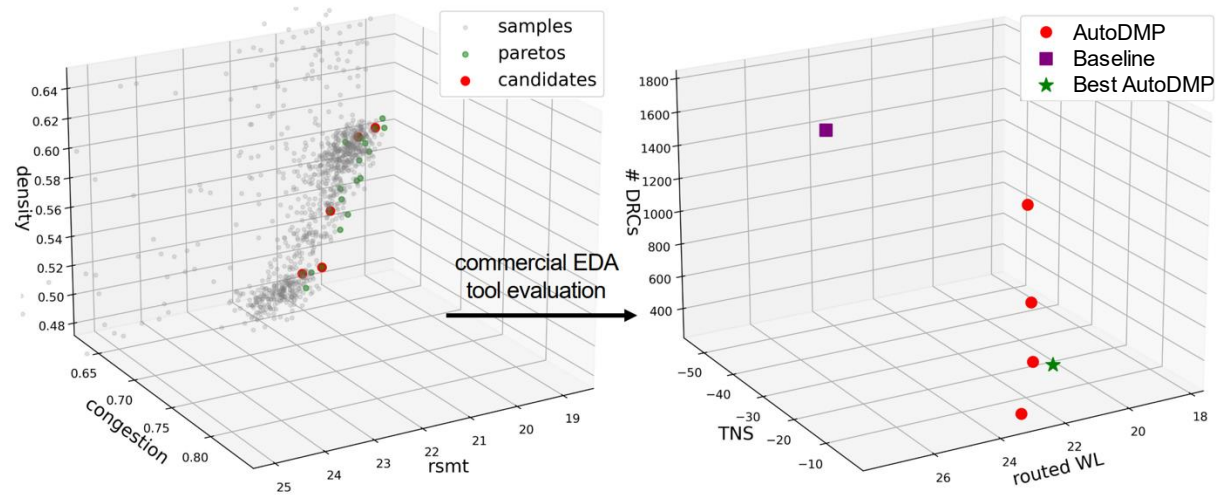
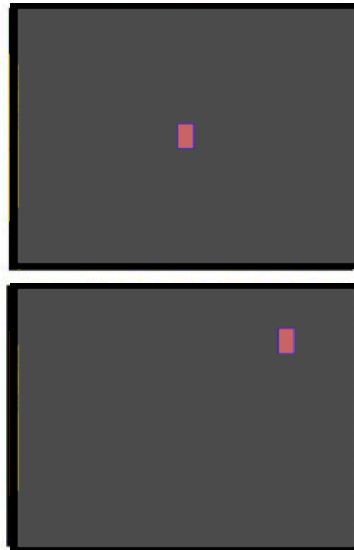
Macro placement quality is very important for physical design

Placement parameters have a huge impact on macro placement

Multi-objective Bayesian optimization: wirelength, congestion, density

Find better macro placement with open-source GPU accelerated placement tools

Parameter	Search Range
*horiz. initial position	[0.2, 0.8] (%)
*vert. initial position	[0.2, 0.8] (%)
*horiz. macro halo	technology dep.
*vert. macro halo	technology dep.
target density d_{target}	$[a_{\text{util}} - 0.2, a_{\text{util}}]$ (%)
density weight	$[1e^{-6}, 1.0]$
smooth HPWL model	{LSE, WA}
smooth HPWL initial γ_0	[0.10, 0.50]
GD initial LR lr_0	$[1e^{-4}, 1e^{-2}]$
GD LR decay	[0.99, 1.0]
GD optimizer	{Adam, Nesterov}
# horiz. global bins	{256, 512, 1024, 2048}
# vert. global bins	{256, 512, 1024, 2048}
λ update lower coeff. L	[0.90, 0.99]
λ update upper coeff. U	[1.01, 1.15]
λ update Δ HPWL _{REF}	$[1.5e^5, 5.5e^5]$



AI fixes design rule check(DRC) errors

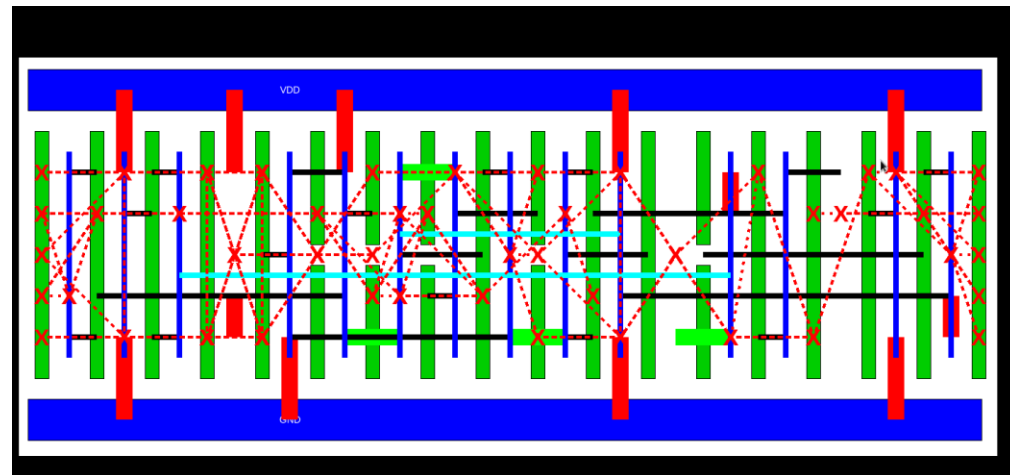
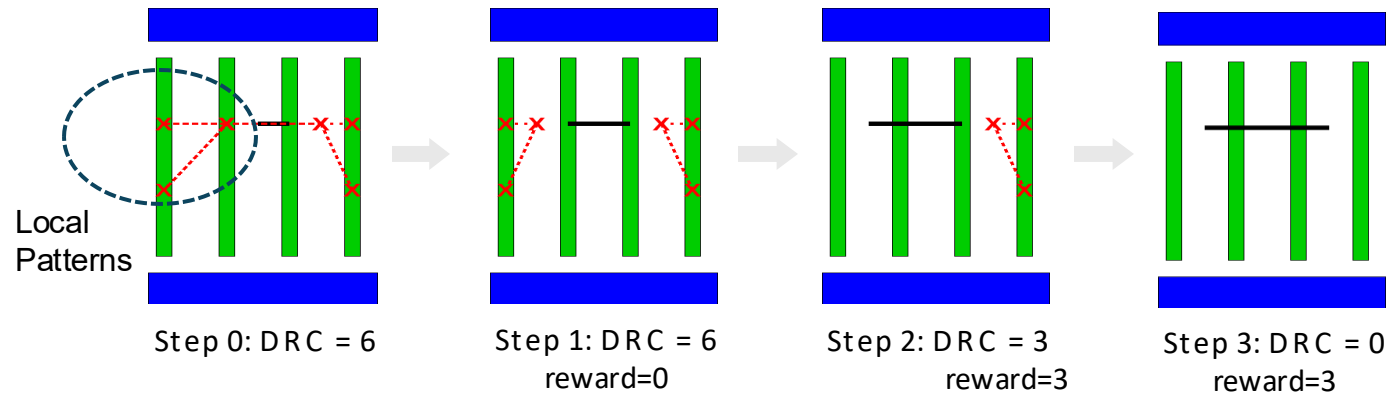
Too many DRC rules to consider for cell layout
RL agent learns to fix DRC automatically

Action: adding additional M0 grid to reduce DRCs

State: current layout images

Reward: DRC reduction

Used in NVCell layout generator in production

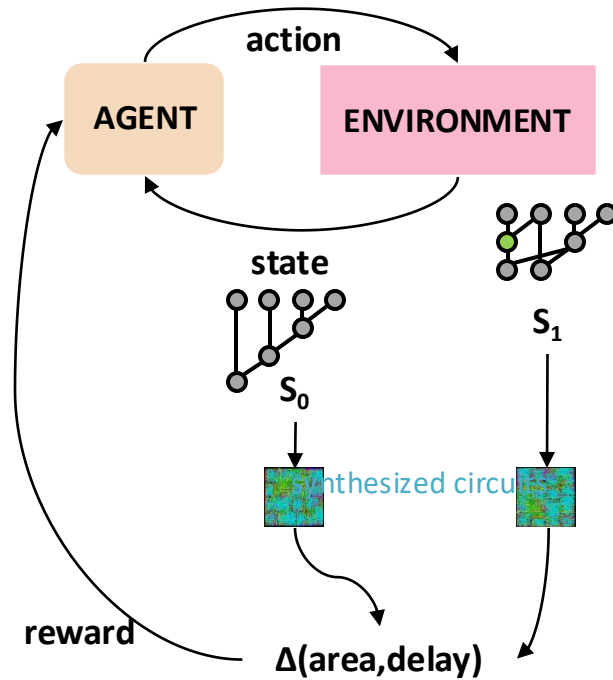


AI optimizes datapath structure

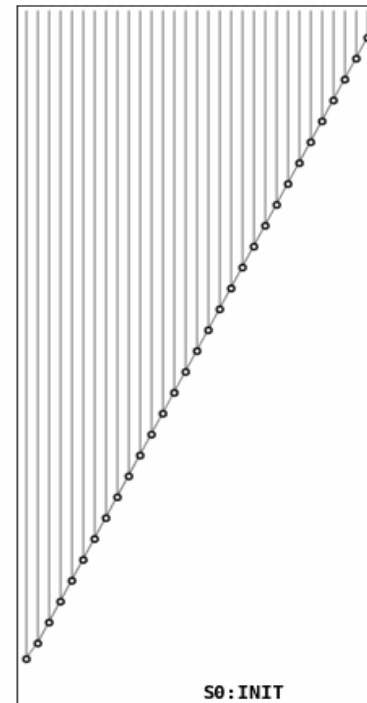
Datapath synthesis important for GPU

Optimize prefix adder structure with **Reinforcement Learning** for better delay and area tradeoff

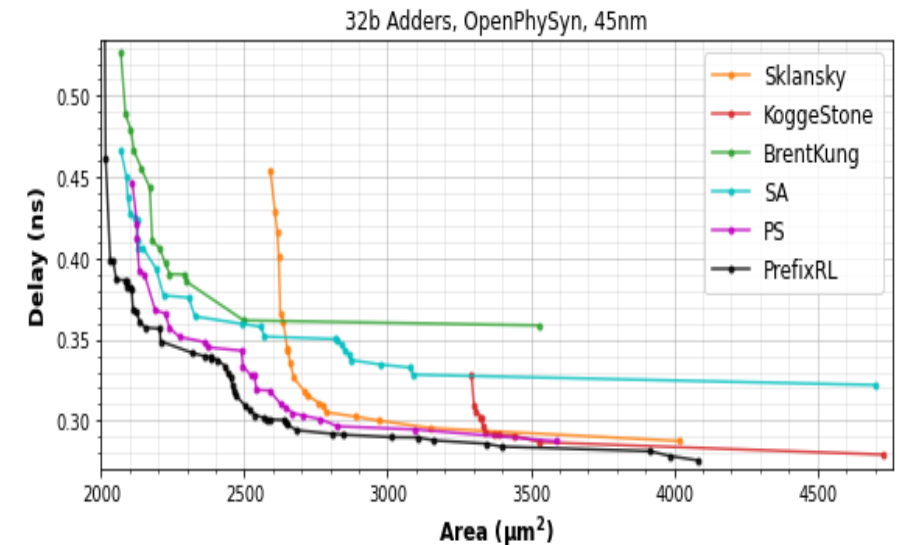
Reward computed by open-source EDA tools



Deep Q learning
Circuit synthesis in-the-loop



Action Space
Add/Delete prefix graph nodes



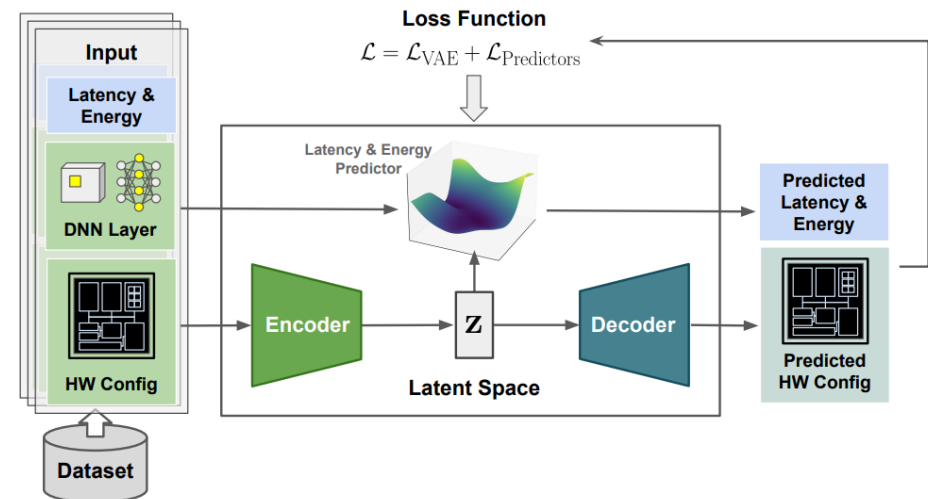
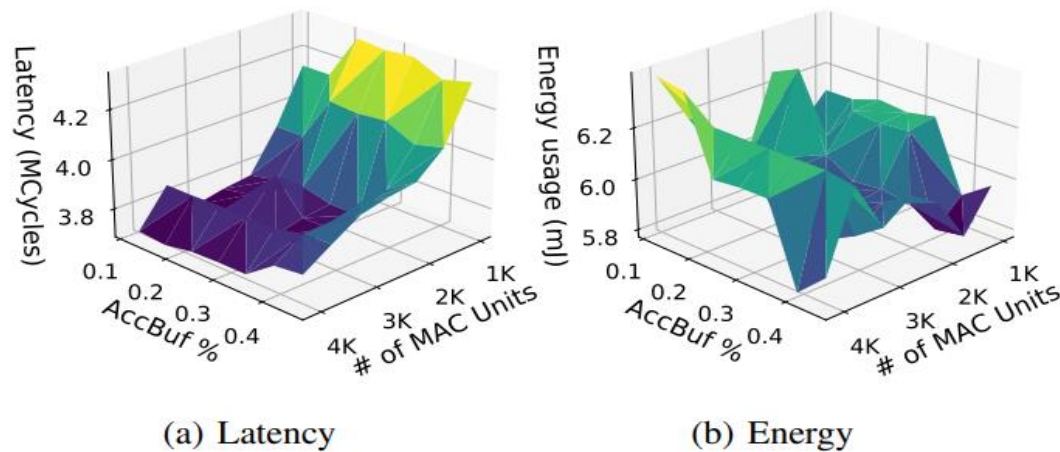
PrefixRL achieves better results than well known adder architectures

AI finds better design parameters

Irregular landscape of neural network accelerator design space

Optimize on the latent space (reduced dim, smooth) learned using VAE

6.8X sample efficiency and 5% better performance



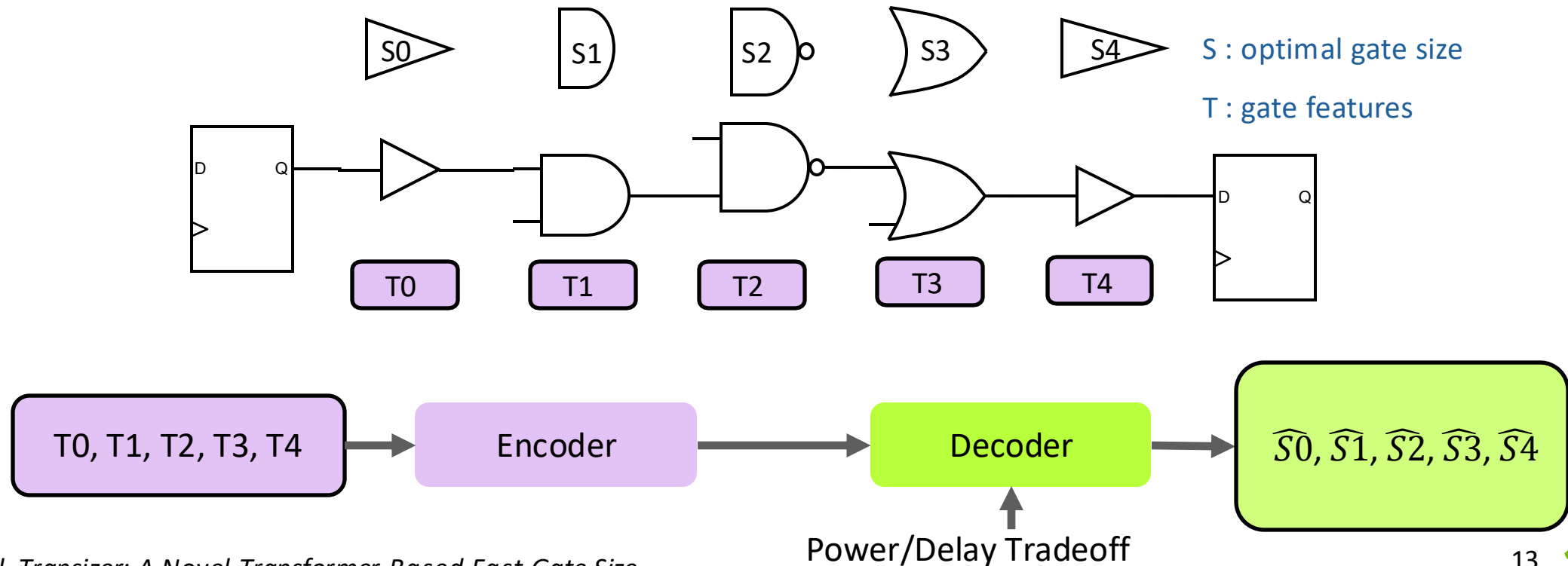
AI generates optimal gate sizes

Timing/power optimization such as gate sizing affects scalability of PD tools

Model a path of gates as a sequence, generate optimized gate sizes using **Transformer**

Trained on tool optimized netlist dataset

100X – 1000X speedup compared to traditional optimization with similar PPA



LLMs provide unprecedented design assistance

Coding Assistance → Generating design and verification code (RTL, SVA, testbenches, EDA scripts, tools scripts, and configs)

- Generate code from specifications

- Generate code for auxiliary design tasks such as assertions, comments, etc.

- Generate lower-level programs from higher-level descriptions

- Generate scripts for specific tasks (VLSI, Verification)

- Transform code for more efficient implementation

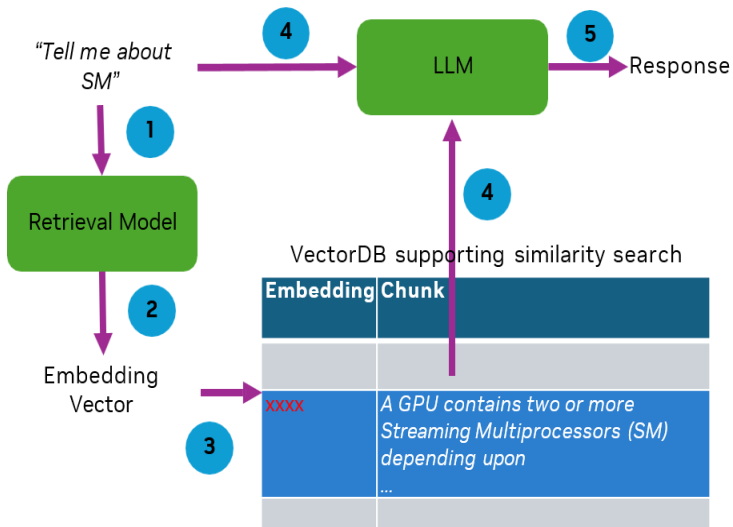
Know-how Assistance → Generating insights, knowledge, and ideas. Answer questions about designs, infrastructures, tools, flows, HW domains, etc.

Analysis Assistance → Summarization, report/log analysis, visualization of design and related data, check rule violations, etc.

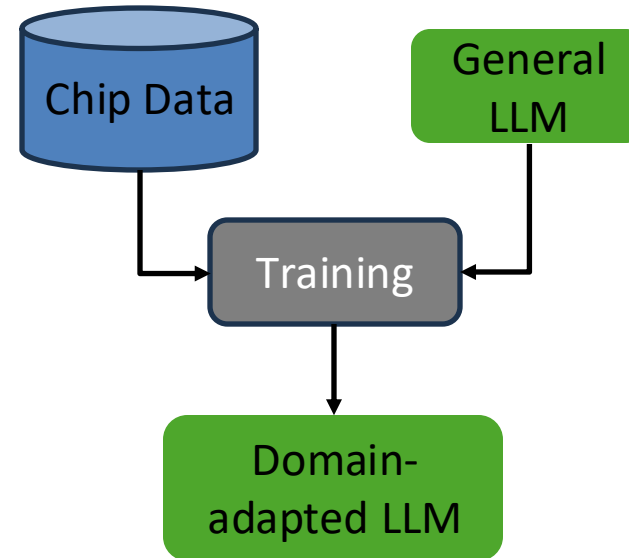
Optimization Assistance → Optimize for design and verification: coverage closure, recipe optimization, architecture exploration, etc.

Debug Assistance → Triage and fix a design problem, e.g. regression failures for logic design and timing failures for physical design.

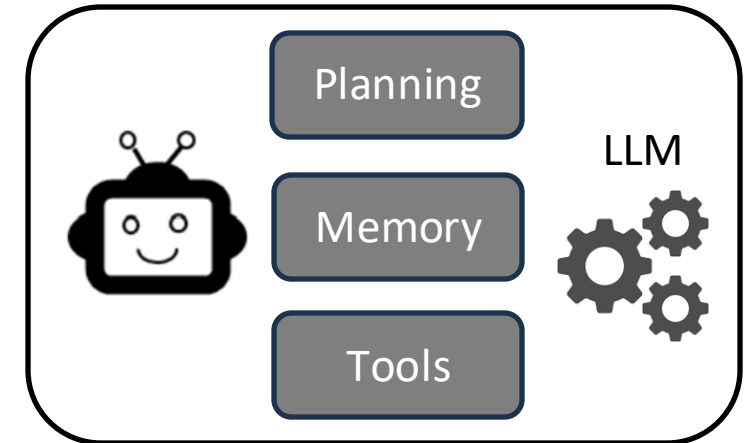
Make LLM learn to do chip design



Prompt engineering, In-Context Learning, RAG



Parameter Training



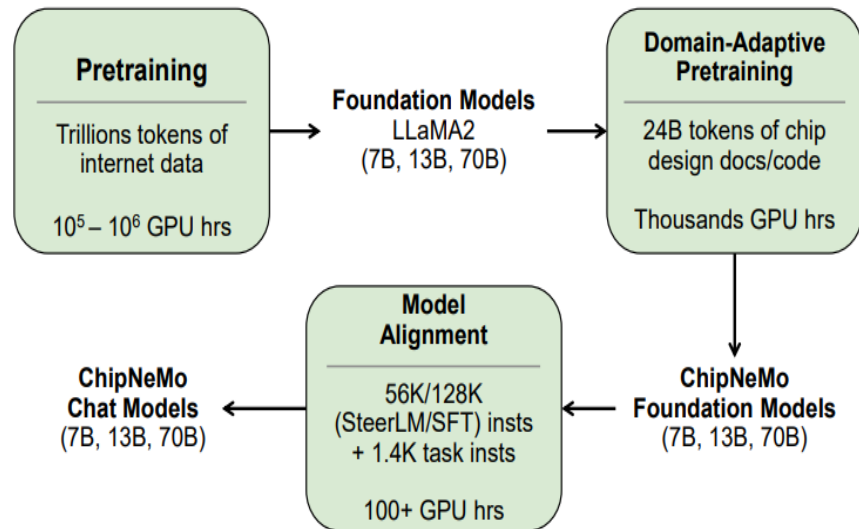
Agent

ChipNeMo : domain-adapted model for chip design

Incorporate domain knowledge with continued pretraining using domain data

- Answer questions about designs, infrastructures, tools, flows, HW domains, etc.
- Generate scripts for specific tasks (VLSI)
- Summarize bug reports, predict assignment

Domain-adapted models have better performance



Question:

Prompt:
BugId : 002
Synopsis : Request power/current measurement on fabrics using fabric test module
Module : Silicon - DFT
Description :
Hi all,
We want to evaluate silicon current (power) consumed by fabric macros in a chip. This will help us correlate this power to pre-Si power estimates. Here are the conditions we want to look at for this data:
Fabric test module activity: 0/25/50/75/100%
Corners:
typical_low_voltage
typical_high_voltage
We want to isolate fabric current (power) as much as possible, so we want to enable only the logic needed for fabric test modules and the fabrics themselves.
If this information can be broken down per module (there are 6 in the chip), that will help.
Severity : 5-Performance
Priority : 5-Normal
Comments :
#1 : Author : User A
Is this a requirement only for current or for future chips as well ?
#2 : Author : User B
There will likely be interest in future chips to perform this exercise as we push the boundaries of fabric power density for better area.
#3 : Author : User A
What is needed in this bug ?
@User B It is not clear.
Can you list the specific needs with timelines and priority

Technical Summary:
In NVIDIA's internal bug management system, a bug has been filed regarding the power and current measurements of the fabrics in the latest chip. The request is to evaluate the silicon current (power) consumed by the fabric macros in the latest chip and to correlate this power to pre-Si power estimates. The conditions for the measurements are the fabric test module activity at 0/25/50/75/100% and the corners are typical_low_voltage and typical_high_voltage. The data needs to be broken down per module (there are 6 in the latest chip). Only the logic needed for fabric test modules and the fabrics themselves need to be enabled.

Managerial Summary:
User A asks User B if the need for evaluating the silicon current of fabric macros in a chip is only for that specific chip or for future ones as well. User B replies that there will likely be interest in future chips to perform this exercise as they push the boundaries of fabric power density for better area. User A then asks what is needed in this bug and asks for specific needs with timelines and priority.

Task Assignment:
User B

Timing agent analyzes multiple timing reports

Task: multi-corner multi-mode (MCMM) timing debugging

QueryTask



Challenges:

- Multiple Timing Reports that provide various timing information (i.e., Max, Xtalk, etc)
- 440k lines in a report, 8k timing paths with relations between timing paths

Solution:

- Knowledge graph for timing report relationship
- Agent generates code to query long report



Agents perform better than designers.



Give me the Path ID with min slack, among paths with starting rising edge in both arcs

Thought: I need iterate through the tables and check the start transition edges of both arcs, keeping the ID of the one with minimum slack in memory

Please execute this code and provide the output

```
min_slack = float('inf')
min_slack_path_id = ""
for table in current_working_dict:
    if re.search(r"r", table['Summary']
['Data_Start_transition']) and re.search(r"r",
table['Summary']['Clock_Start_transition']):
        slack = float(table['Summary']['slack'])
        if slack < min_slack:
            min_slack = slack
            min_slack_path_id = table['Summary']['Path ID']

result = {"min_slack_path_id": min_slack_path_id,
"min_slack": min_slack}
```

```
result =
{"min_slack_path_id": 283008, "min_slack": -0.2466}
```

Final Answer:

The Path ID with minimum slack is 283008 with slack of -0.2466

VerilogCoder plan and implement Verilog code

Task: Generate Verilog from the specification

Challenges:

- Misunderstand specification
- Incorrect code implementation

Solution:

- Two-stage flow: Planning and implementation
- Leverage special knowledge-base and tools

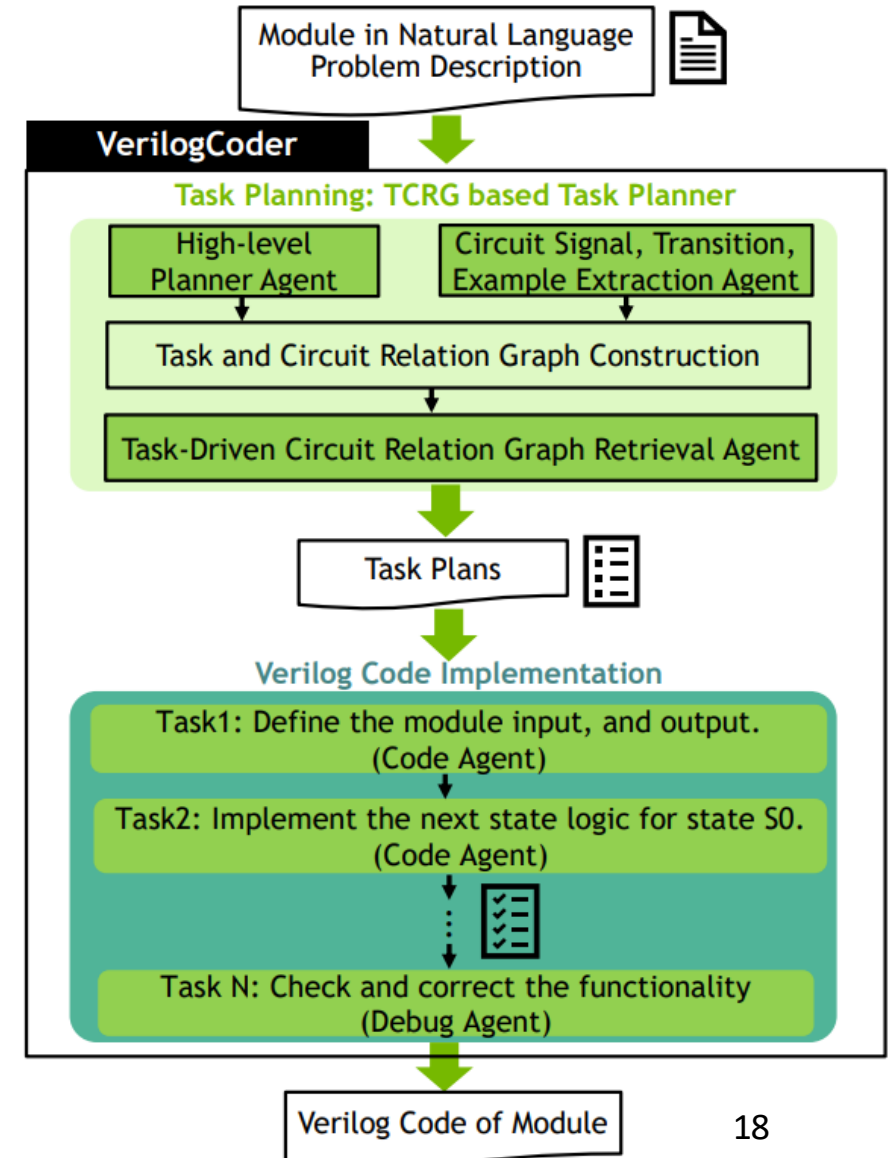
Task Planning

Task-Driven Circuit Relation Graph (TCRG)

Code Implementation

AST-guided waveform debugging tool

Improve success rate from 60% to 94%



```
Mate Terminal
Mate Terminal
Mate Terminal

(/home/scratch.chiatungh_nvresearch/nvcell env/llm env) bash-4.2$ python hardware_agent/examples/verilog_eval_agent/Verilog_eval_task_flow_ver1.py
reading /home/scratch.chiatungh_nvresearch/hardware-agent-marco/hardware_agent/examples/verilog_testcases/verilog-eval-v2/dataset_dumpall/ Prob139_2013_q2bfsm_prompt.txt
reading /home/scratch.chiatungh_nvresearch/hardware-agent-marco/hardware_agent/examples/verilog_testcases/verilog-eval-v2/dataset_dumpall/ Prob139_2013_q2bfsm_ref.sv
reading /home/scratch.chiatungh_nvresearch/hardware-agent-marco/hardware_agent/examples/verilog_testcases/verilog-eval-v2/dataset_dumpall/ Prob139_2013_q2bfsm_test.sv
{'llm_class': <class 'adlrchat.langchain.LLMGatewayChat'>, 'llm_kwargs': {'model_name': 'gpt-4', 'temperature': 0.0, 'top_p': 1.0}}
case manager length = 1
current test is 2013_q2bfsm
current task id is 2013_q2bfsm
current test is 2013_q2bfsm
register tool sequential_flipflop_latch_identify_tool <function sequential_flipflop_latch_identify_tool at 0x7f65abdac040> caller: <autogen.agentchat.assistant_agent.AssistantAgent object at 0x7f65a93ad1b0> executer: <autogen.agentchat.user_proxy_agent.UserProxyAgent object at 0x7f65a93ad390>
hat.user_proxy_agent.UserProxyAgent object at 0x7f65a93ad390>
Hardware Agent Initialized 1 proxy, 0 rag proxy, 2 assistants
Hardware Agent Initialized 1 proxy, 0 rag proxy, 1 assistants
user (to chat_manager):

You are a Verilog RTL designer that can break down complicated implementation into subtasks implementation plans.

[Example Begin]

### Problem

I would like you to implement a module named TopModule with the following
interface. All input and output ports are one bit unless otherwise
specified.

- input  clk
- input  reset
- input  in (8 bits)
- output out (8 bits)

The module should implement an 8-bit registered incrementer. The 8-bit
input is first registered and then incremented by one on the next cycle.

Assume all sequential logic is triggered on the positive edge of the
clock. The reset input is active high synchronous and should reset the
output to zero.

### Solution

module TopModule
(
    input logic      clk,
    input logic      reset,
    input logic [7:0] in,
    output logic [7:0] out
);

// Sequential logic

logic [7:0] reg_out;

always @( posedge clk ) begin
    if ( reset )
        reg_out <= 0;
    else
        reg_out <= in;
end

// Combinational logic

logic [7:0] temp_wire;

always @(*) begin
    temp_wire = reg_out + 1;
end
```

Conclusion

- Chip enables AI, and AI assists chip design
- AI enables fast and cross-stage design analysis
- AI performs scalable and better design optimization
- LLM models and agents provide coding, know-how, optimization, debug and analysis assistance, significantly improve chip design productivity

